For the complete list, you can refer to the Spring Boot documentation (https://spring.io/projects/spring-boot).

Configuration data files can be further prioritized, starting with the highest priority:

1. Profile-specific application properties from application-{profile}.properties and application-{profile}.yml files packaged outside your JAR
2. Application properties from application.properties and application.yml files packaged outside your JAR
3. Profile-specific application properties from application-{profile}.properties and application-{profile}.yml files packaged inside your JAR
4. Application properties from application.properties and application.yml files packaged inside your JAR

The beauty of property handling in Spring is that you don't need to know the specific property source to get a value: the `Environment` abstraction lets you access any property defined in any source through a unified interface. And if the same property is defined in multiple sources, it returns the one with the highest priority. You can even add your own custom sources and assign a priority to them.

NOTE Spring Framework has built-in support for properties defined according to the `Properties` format. On top of that, Spring Boot adds support for defining properties using the YAML format. YAML is a superset of JSON that provides more flexibility than the simpler `Properties` format. The official website describes YAML as "a human-friendly data serialization language for all programming languages" (https://yaml.org). Feel free to choose either approach in your applications. All the examples in the book will use YAML.

USING APPLICATION PROPERTIES